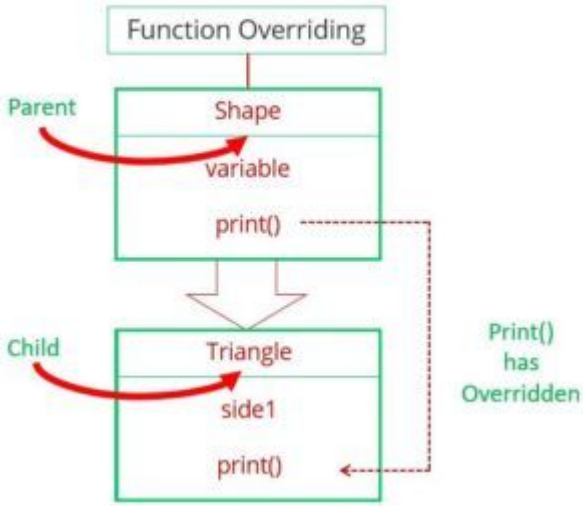
RunTime Polymorphism

Function Overriding

When overriding a method, the behavior of the method is changed for the derived class.

A class may need to override the default behavior provided by its base class.

- Provide behavior specific to a derived class
- Extend the default behavior
- Restrict the default behavior
- Improve performance



Function OverRiding

```
class Base {
    public:
      void print() {
          // code
      }
};

class Derived : public Base {
    public:
      void print() {
          // code
      }
};

int main() {
    Derived derived1, derived2;

    derived1.print();

    derived2.Base::print();

    return 0;
}
```

Function OverRiding

```
class Base {
    public:
      void print() {
          // code
      }
};

class Derived : public Base {
    public:
      void print() {
          // code
          Base::print();
      }
};

int main() {
    Derived derived1;
    derived1.print();
    return 0;
}
```

When I override my parent's methods

Function Overriding

```
#include <iostream>
using namespace std;

class base_class
{
public:
   void findArea(int side)
      {cout << "Base::The area is: "<<(side * side) << "\n\n"; }
   void findArea(int x, int y)
      {cout << "Base::The area is: "<<(x * y) << "\n\n"; }
};

class derived_class : public base_class
{
public:
 void findArea(int side)
      { cout << "Derived::The area is: "<<(side * side) << "\n\n"; }
void findArea(int x, int y)
      {cout << "Derived::The area is: "<<(x * y) << "\n\n"; }

};
```

```
int main()
{
      derived_class obj;
      obj.findArea(5);
      obj.findArea(5,4);
      obj.base_class::findArea(6
);
      return 0;

}
```

Function Overriding

Inheritance should be there. Function overriding cannot be done within a class. For this we require a derived class and a base class.

Function that is redefined must have exactly the same declaration in both base and derived class, that means same name, same return type and same parameter list.

Function Overloading VS Function Overriding

Inheritance: Overriding of functions occurs when one class is inherited from another class. Overloading can occur without inheritance.

Function Signature: Overloaded functions must differ in function signature ie either number of parameters or type of parameters should differ. In overriding, function signatures must be same.

Scope of functions: Overridden functions are in different scopes; whereas overloaded functions are in same scope.

Behavior of functions: Overriding is needed when derived class function has to do some added or different job than the base class function. Overloading is used to have same name functions which behave differently depending upon parameters passed to them.

Early Binding

Connecting the function call to the function body is called **Binding**.

When it is done before the program is run, its called **Early Binding or Static Binding or Compile-time Binding**.

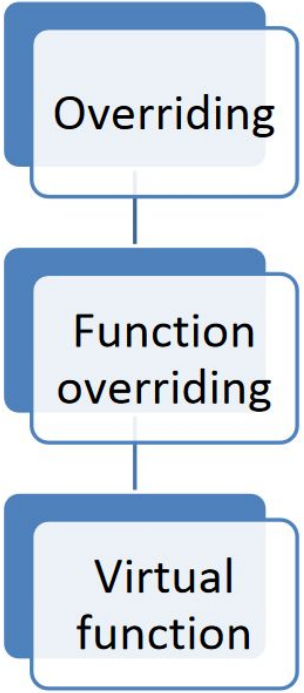
Compiler gives preference to object type.

Base class object will call base version of the function and derived class's object will call the derived version of the function. This is called early binding

Early Binding

```
class A {
public:
void display() {
cout<<"Base class"; } };

class B:public A {
public:
void display() {
cout<<"Derived Class"; } };

int main() {
A obj1;
B obj2;
obj1.display();
obj2.display();
return 0; }
```

Run time / Dynamic / late Binding polymorphism



Run time / Dynamic / late Binding polymorphism

This refers to the entity which changes its form depending on circumstances at runtime. This concept can be adopted as analogous to a chameleon changing its color at the sight of an approaching object.

Method Overriding uses runtime Polymorphism.

It is also called Late Binding.

Early Binding

```
#include <iostream>
using namespace std;

class Base {
public:
void print() {
cout << "base class print()" << endl; }};

class Derived : public Base {
public:
void print() {
cout << "derived class print()" << endl;}};
```

```
int main() {
Derived derived1;
Base* base1 = &derived1;
base1->print();}
```

Output

```
/tmp/7OMUjpQe9c.o
base class print()
```

Problem

We aimed a base-class pointer at a derived-class object, then invoked member function print through that pointer. But the type of the handle(pointer) determines which class's functionality to invoke. In that case, the Base class pointer invoked the member function print on the derived-class object, even though the pointer was aimed at a derived-class object that has its own customized print function.

Solution

With virtual functions, the type of the object being pointed to, not the type of the handle, determines which version of a virtual function to invoke.

Virtual Function

A virtual function is a member function that you expect to be redefined in derived classes. (Overridden)

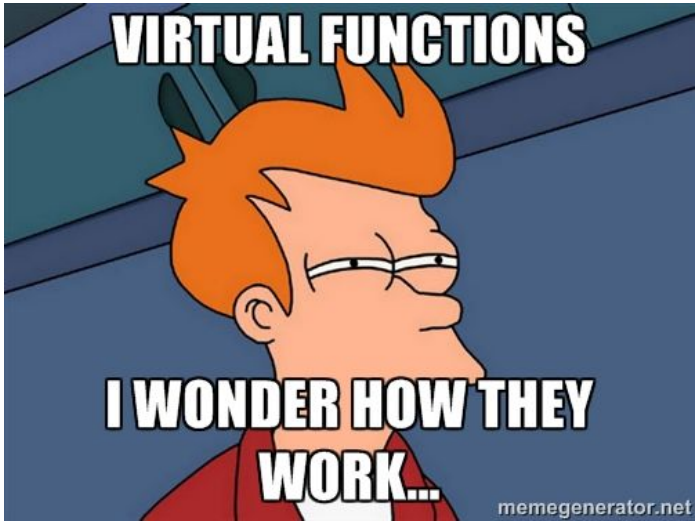
When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.

Virtual Function

They are mainly used to achieve Runtime polymorphism

Functions are declared with a virtual keyword in base class.

The resolving of function call is done at Run-time.



Rules for Virtual Function

They Must be declared in public section of class.

Virtual functions cannot be static and also cannot be a friend function of another class.

They are always defined in base class and overridden in derived class. It is not mandatory for derived class to override (or re-define the virtual function), in that case base class version of function is used.

Late Binding

```
#include <iostream>
using namespace std;

class Base {
public:
virtual void print() {
cout << "base class print()" << endl; }};

class Derived : public Base {
public:
void print() {
cout << "derived class print()" << endl;}};
```

```
int main() {
Derived derived1;
Base* base1 = &derived1;
base1->print();}
```

Output

```
/tmp/7OMUjpQe9c.o
derived class print()
```

```cpp
#include <iostream>
using namespace std;

class Base {
public:
virtual void print() {
cout << "base class print()" << endl; }
void show() {
cout << "Base class Show()" << endl; }
};

class Derived : public Base {
public:
void print() {
cout << "derived class print()" << endl;}
void show() {
cout << "derived class Show()" << endl; }
};
int main() {
Derived derived1;
Base* base1 = &derived1;
base1->print();
base1->show();
}
```

Output

```
/tmp/7OMUjpQe9c.o
derived class print()
Base class Show()
```

Virtual Function

Runtime polymorphism is achieved only through a pointer (or reference) of base class type

Late binding(Runtime) is done in accordance with the content of pointer (i.e. location pointed to by pointer).

Early binding(Compile time) is done according to the type of pointer.

Virtual Function

When calling a function using pointers or references, the following rules apply:

- A call to a virtual function is resolved according to the underlying type of object for which it is called.
- A call to a nonvirtual function is resolved according to the type of the pointer or reference.
- The virtual keyword can be used when declaring overriding functions in a derived class, but it is unnecessary; overrides of virtual functions are always virtual.

Late Binding

```
#include <iostream>
#include <string>
class Employee;
void HeadQuarter(Employee * e); // Forward Declaration
using namespace std;

class Employee {
Protected:     string name;  double baseSalary;
public:
    Employee(const string& name, double baseSalary) :
name(name), baseSalary(baseSalary) {}
 void work()     {cout<<"I am Working"<<endl;}
 string getname(){ return name;}
 virtual double calculateMonthlySalary() { return baseSalary; }
```

Late Binding

```
class Programmer : public Employee {
public:
    Programmer(const string& name, double baseSalary) :
Employee(name, baseSalary) {}
    void work()
    {
        cout<<"I am Programming"<<endl;
    }
    double calculateMonthlySalary() override {
        return baseSalary + 500;
    }
};
```

Late Binding

```
class Developer : public Employee {
public:
    Developer(const string& name, double baseSalary) :
Employee(name, baseSalary) {}
   void work()
    {
        cout<<"I am Developing"<<endl;
    }
    double calculateMonthlySalary() override {
        return baseSalary + 1000;
    }
};
```

Late Binding

```
int main() {
    Employee* john = new Programmer("John", 3000);

    Employee* emp;
    Developer Dev1("Ali", 4000);
    emp=&Dev1;
```

Late Binding

```
int main() {
    Employee* john = new Programmer("John", 3000);

    Employee* emp;
    Developer Dev1("Ali", 4000);
    emp=&Dev1;


cout << "Monthly salary "  << john-> calculateMonthlySalary() <<
endl;
cout << "Monthly salary "  << emp->calculateMonthlySalary() <<
endl;
```

Late Binding

```
int main() {
    Employee* john = new Programmer("John", 3000);
    Employee* emp;
    Developer Dev1("Ali", 4000);
    emp=&Dev1;

    HeadQuarter(&Dev1);
    HeadQuarter(john);
    delete john;

    return 0;
}
```

Late Binding

```
void HeadQuarter(Employee * e)
{
    /* Polymorphism allows different objects to be treated
uniformly through a common interface, even if they belong
to different classes. This facilitates code reuse,
flexibility, and extensibility in software design*/

    cout<<"Final Reporting"<<endl;
   // e->work();
    cout<<"Employee Name :" <<e->getname();
    cout<<" "<<e->calculateMonthlySalary()<<endl;
}
```

Diamond Problem

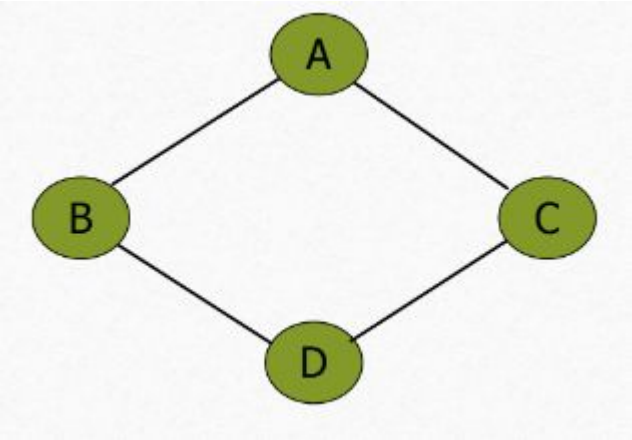
Problem : They are too expensive :p

Diamond Problem

In case of hybrid inheritance, a Diamond problem may arise. The “dreaded diamond” refers to a class structure in which a particular class appears more than once in a class’s inheritance hierarchy.

Common dangerous pattern: "The Diamond"

- Classes B and C extend A
- Class D extends A and B
- Class D extends A and C



Diamond Problem

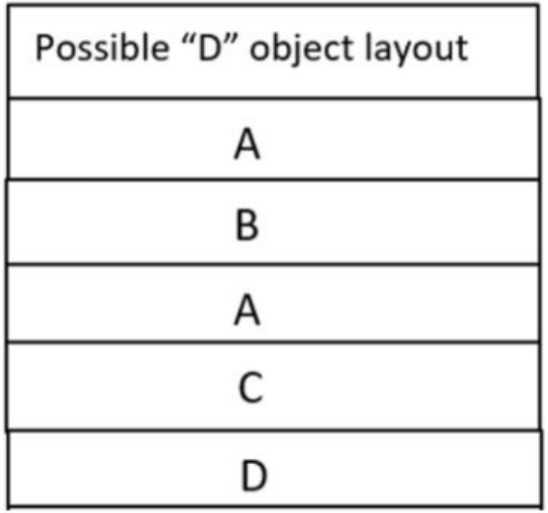
```
#include <iostream>
using namespace std;

class A{
public:
int a;};

class B : public A{
public:
int b;};

class C : public A{
public:
int c;};

class D : public B , public C{
public:
int d;};
```

```
int main(){
D d;
d.a;
return 0;}
```

Diamond Problem

“A” is inherited two times in D means an object of class “D” will contain two attributes of “a” (D::C::a and D::B::a).

```
/tmp/7OMUjpQe9c.cpp: In function 'int main()':
/tmp/7OMUjpQe9c.cpp:22:3: error: request for member 'a' is ambiguous
   22 | d.a;
      |   ^
/tmp/7OMUjpQe9c.cpp:7:5: note: candidates are: 'int A::a'
    7 | int a;};
      |     ^
/tmp/7OMUjpQe9c.cpp:7:5: note:                 'int A::a'
```



Supposed memory layout of class “D” object

How to Resolve?

Solution to Diamond Problem – Virtual Inheritance

- What happens without virtual inheritance:

- You want:Achievable with virtual inheritance

Virtual Base Class \ Virtual Inheritance

Virtual base classes offer a way to save space and avoid ambiguities in class hierarchies that use multiple inheritances.

When a base class is specified as a virtual base, it can act as an indirect base more than once without duplication of its data members.

A single copy of its data members is shared by all the base classes that use virtual base.

Virtual Base Class \ Virtual Inheritance

Virtual base classes offer a way to save space and avoid ambiguities in class hierarchies that use multiple inheritances.

When a base class is specified as a virtual base, it can act as an indirect base more than once without duplication of its data members.

A single copy of its data members is shared by all the base classes that use virtual base.

Diamond Problem

```
#include <iostream>
using namespace std;

class A{
public:
int a;};

class B : virtual public A{
public:
int b;};

class C : virtual public A{
public:
int c;};

class D : public B , public C{
public:
int d;};
```

```
int main(){
D d;
d.a=10;
return 0;}
```

Diamond Problem

```
#include <iostream>
using namespace std;

class LivingThing{
public:
void breathe()
    {cout<<"I am breathing as a Living being";
    }};

class Animal : virtual public LivingThing{
public:
};

class Reptile : virtual public LivingThing{
public:
void crawl()
    {cout<<"I am crawling as a Reptile"; } };
```

```
class Snake : public Animal ,
public Reptile{};

int main(){
Snake s1;
s1.breathe();
s1.crawl();
return 0;}
```

Diamond Problem

```
#include <iostream>
using namespace std;

class LivingThing{
public:
void breathe()
    {cout<<"I am breathing as a Living being";
    }};

class Animal : virtual public LivingThing{
public:
void breathe()
    {
        cout<<"I am breathing as an Animal";
    }};

class Reptile : virtual public LivingThing{
public:
void crawl()
    {cout<<"I am crawling as a Reptile"; }};
```

```
class Snake : public Animal , public
Reptile{};

int main(){
Snake s1;
s1.breathe();
s1.crawl();
return 0;}
```

Diamond Problem

```
#include <iostream>
using namespace std;

class LivingThing{
public:
void breathe()
    {cout<<"I am breathing as a Living being";
    }};

class Animal : virtual public LivingThing{
public:
void breathe()
    {
        cout<<"I am breathing as an Animal";
    }};

class Reptile : virtual public LivingThing{
public:
void crawl()
    {cout<<"I am crawling as a Reptile"; }
    void breathe()
    {  cout<<"I am breathing as an Reptile";
    } };
```

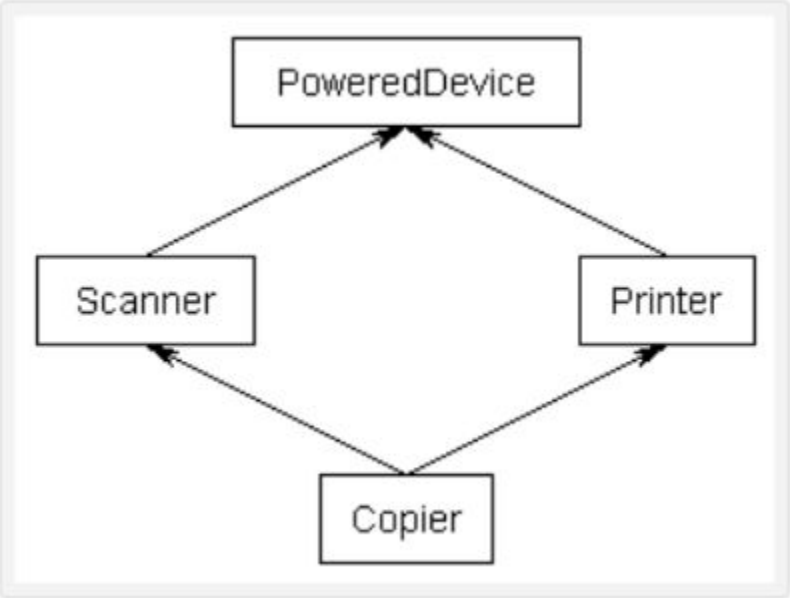
```
class Snake : public Animal , public
Reptile{};

int main(){
Snake s1;
s1.breathe();
s1.crawl();
return 0;}
```

Diamond Problem

```
#include <iostream>
using namespace std;

class LivingThing{
public:
void breathe()
    {cout<<"I am breathing as a Living being";
    }};

class Animal : virtual public LivingThing{
public:
void breathe()
    {
        cout<<"I am breathing as an Animal";
    }};

class Reptile : virtual public LivingThing{
public:
void crawl()
    {cout<<"I am crawling as a Reptile"; }
    void breathe()
    {  cout<<"I am breathing as an Reptile";
    } };
```

```
class Snake : public Animal , public
Reptile{
    public:  void breathe()
    {
        cout<<"I am breathing as an
Snake";
    }
};

int main(){
Snake s1;
s1.breathe();
s1.crawl();
return 0;}
```

Diamond Problem

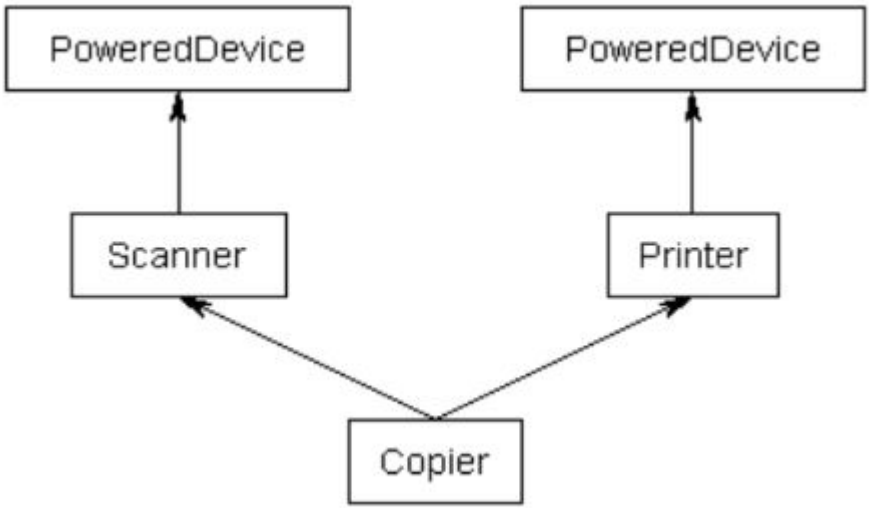


```
1  #include <iostream>
2  using namespace std;
3  class PoweredDevice{
4  public:
5  PoweredDevice(int power){
6  cout << "PoweredDevice: " << power << '\n';}};
7
8  class Scanner: public PoweredDevice{
9  public:
10 Scanner(int scanner, int power): PoweredDevice(power) {
11 cout << "Scanner: " << scanner << '\n'; }};
12
13 class Printer: public PoweredDevice{
14 public:
15 Printer(int printer, int power): PoweredDevice(power) {
16 cout << "Printer: " << printer << '\n'; }};
17
18 class Copier: public Scanner, public Printer{
19 public:
20 Copier(int scanner, int printer, int power): Scanner(scanner, power), Printe
       (printer, power) {
21 }};
22 int main()
23 {
24 Copier copier(1, 2, 3);
25 return 0;}
```

Diamond Problem

If you were to create a Copier class object, by default you would end up with two copies of the PoweredDevice class -- one from Printer, and one from Scanner. This has the following structure:

```
PoweredDevice: 3
Scanner: 1
PoweredDevice: 3
Printer: 2
```



```cpp
#include <iostream>
using namespace std;
class PoweredDevice{
public:
PoweredDevice(int power){
cout << "PoweredDevice: " << power << '\n';}};

class Scanner: virtual public PoweredDevice{
public:
Scanner(int scanner, int power): PoweredDevice(power) {
cout << "Scanner: " << scanner << '\n'; }};

class Printer: virtual public PoweredDevice{
public:
Printer(int printer, int power): PoweredDevice(power) {
cout << "Printer: " << printer << '\n'; }};

class Copier: public Scanner, public Printer{
public:
Copier(int scanner, int printer, int power): PoweredDevice(power), Scanner
    (scanner, power), Printer(printer, power) {
}};
int main()
{
Copier copier(1, 2, 3);
return 0;}
```

```
class Scanner: virtual public PoweredDevice

{

public:

Scanner(int scanner, int power): PoweredDevice(power) // this
line is required to create Scanner objects, but ignored in this
case because we are working with copier object

{

std::cout << "Scanner: " << scanner << '\n'; }};
```

```
class Printer: virtual public PoweredDevice

{

public:

Printer(int printer, int power): PoweredDevice(power) // this line is required
to create Printer objects, but ignored in this case because we are working
with copier object

{

std::cout << "Printer: " << printer << '\n';

}};
```

```
class Copier: public Scanner, public Printer

{

public:

Copier( int scanner ,int printer, int power) : PoweredDevice(power), Scanner(scanner,
power),Printer(printer, power)

// PoweredDevice is constructed here updated value is provided by grandchild

{

}

}
```

Diamond Problem

Now, when you create a Copier class object, you will get only one copy of PoweredDevice per Copier that will be shared by both Scanner and Printer.

The Copier constructor is responsible for creating PoweredDevice.

In general, it is not allowed to call the grandparent’s constructor directly, it has to be called through parent class. It is allowed only when ‘virtual’ keyword is used.

```cpp
#include<iostream>
using namespace std;
class Person {
public:
    Person(int x)  { cout << "Person::Person(int ) called" << endl;   }
    Person()     { cout << "Person::Person() called" << endl;   }
};

class Faculty : virtual public Person {
public:
    Faculty(int x):Person(x)   {
       cout<<"Faculty::Faculty(int ) called"<< endl;
    }
};

class Student : virtual public Person {
public:
    Student(int x):Person(x) {
        cout<<"Student::Student(int ) called"<< endl;
    }
};

class TA : public Faculty, public Student  {
public:
    TA(int x):Student(x), Faculty(x), Person(x)   {
        cout<<"TA::TA(int ) called"<< endl;
    }
};

int main()  {
    TA ta1(30);
}
```